



Conditions et Boucles





Instructions de test

Le langage C permet trois types de tests différents : **simple**, **avec alternative**, et **multiple**.

Pour chacun, on distingue deux parties :

- Une ou plusieurs conditions ;
- Un ou plusieurs blocs.

Condition : expression dont la valeur est interprétée de façon booléenne : la condition peut être soit vraie, soit fausse.

Le principe est d'associer une condition à un bloc.

Autrement, si une condition donnée est vraie, alors une certaine partie du programme sera exécutée. Sinon, c'est une autre partie qui sera exécutée, voire aucune partie.

Remarque : une fois l'instruction de test exécutée, le programme reprend son cours normalement.



1. Test simple

- Avec le test simple, on associe une seule condition à un seul bloc de code. Si la condition est vraie, ce bloc est exécuté. Sinon, il n'est pas exécuté.
- Cette instruction est notée if, et sa syntaxe est la suivante :

```
if(condition) {  
instruction 1;  
...  
instruction n;  
}
```



1. Test simple

Le fonctionnement est le suivant. Tout d'abord, l'expression condition est évaluée :

- Si elle est vraie (i.e. si sa valeur est un entier non-nul) alors le bloc est exécuté
- Si elle est fausse (i.e. si sa valeur est zéro) alors le bloc n'est pas exécuté. Comme indiqué précédemment, dans les deux cas, l'exécution du programme continue normalement ensuite.
- exemple : considérons le programme suivant :

```
int x; ...  
if(x>0) {  
printf("La valeur de x est positive\n");  
}  
printf("Le test est terminé\n");
```

```
// x=10  
La valeur de x est positive  
Le test est terminé  
// x=-2  
Le test est terminé
```



2. Test avec alternative

- Avec le test simple, on exécute un bloc seulement si une condition est vraie, et on ne l'exécute pas si la condition n'est pas vraie. Il est possible de proposer un bloc alternatif, à exécuter quand la condition n'est pas vraie (plutôt que de ne rien faire). On parle alors de test avec alternative.
- Ce bloc additionnel est introduit par le mot clé **else**, en respectant la syntaxe suivante :

```
if(condition) {  
  instruction 1;  
  ...  
  Instruction n;  
}  
else {  
  instruction 1;  
  ...  
  instruction n; }  

```



3. Test multiple

- Dans certains cas, on a besoin de comparer une variable donnée à plusieurs valeurs différentes, et non pas une seule comme précédemment.
- exemple : soit une variable entière x. On veut effectuer un traitement différent suivant que la valeur de la variable est 1, 2, 3 ou plus. Si on utilise if, on va devoir procéder par imbrications successives :

```
if(x==1) {  
    ...  
}  
else {  
    if(x==2) { ... }  
    else {  
        if(x==3) { ... }  
        else { ... }  
    }  
}
```



3. Test multiple

- L'instruction `switch` permet d'effectuer le même traitement, mais avec une syntaxe plus compacte. Le mot-clé `switch` reçoit entre parenthèses la variable à tester. Chaque valeur à comparer est ensuite indiquée dans le bloc du `switch`, en utilisant le mot-clé `case`, de la façon suivante :



```
switch(variable) {  
  case valeur_a:  
    instruction 1a;  
    ...  
    instruction na;  
  
  case valeur_b:  
    instruction 1b;  
    ...  
    instruction nb;  
    ...  
}
```

- Le fonctionnement est le suivant. La variable spécifiée va être comparée successivement à chaque valeur proposée, jusqu'à ce qu'il y ait égalité. Si la variable n'est égale à aucune valeur, alors on sort simplement du `switch` sans rien faire de plus. Si on trouve une valeur égale, alors on exécute toutes les instructions situées en-dessous du `case` correspondant.
- Attention : cela inclut non seulement les instructions du `case` associé à la valeur, mais aussi celles de tous les autres `case` suivants !



3. Test multiple

- Or, il arrive fréquemment qu'on désire que seules les instructions placées directement en dessous du case correspondant soient exécutées, mais pas les suivantes. Pour arriver à ce comportement, il faut ajouter l'instruction **break** juste avant le case suivant, comme indiqué ci-dessous :

```
switch(variable) {  
  case valeur_a:  
    instruction 1a;  
    ...  
    instruction na;  
    break;  
  case valeur_b:  
    instruction 1b;  
    ...  
    instruction nb;  
    break;  
  ...  
}
```




3. Test multiple

- Il est possible de définir un cas par défaut, c'est-à-dire de préciser un comportement si la variable ne correspond à aucune des valeurs spécifiées dans les case. On utilise pour cela le mot-clé **default** à la place de case, de la façon suivante :
- À noter que ce cas par défaut se place tout à la fin du switch.

```
switch(x) {  
  case 1:  
    ...  
  break;  
  case 2:  
    ...  
  break;  
  case 2:  
    ...  
  break;  
  default:  
    ...  
}
```



Instructions de répétition

- On distingue trois boucles différentes en C :

tant **que**, **répéter** et **pour**

Nous allons voir qu'en termes d'expressivité, elles ont strictement le même pouvoir (i.e. elles permettent de faire exactement la même chose).

- Cependant, la convention veut qu'on les utilise dans des contextes différents.



Instructions de répétition

Une boucle répétitive se compose de quatre éléments essentiels :

1. Un bloc d'instruction, qui sera exécuté un certain nombre de fois ;
2. Une condition, comme pour les instructions de test. Cette condition porte sur au moins une variable, que l'on appellera la variable de boucle. Il peut y avoir plusieurs variables de boucle pour une seule boucle.
3. Une initialisation, qui concerne la variable de boucle. Cette initialisation peut être directement réalisée par l'instruction de répétition, ou bien laissée à la charge du programmeur.
4. Une modification, qui concerne elle aussi la variable de boucle. Comme pour l'initialisation, elle peut être intégrée à l'instruction de répétition, ou bien laissée à la charge du programmeur.

Si l'un de ces 4 éléments manque, alors la boucle est incorrecte. Cela provoque soit une erreur de compilation, soit une erreur d'exécution, en fonction de l'élément manquant et du type de boucle.



Instructions de répétition

Le principe d'une boucle est le suivant :

- 1) La variable de boucle est initialisée.
- 2) Le bloc d'instructions est exécuté.
- 3) La condition est évaluée.

En fonction de son résultat :

- a. Soit on sort de la boucle.
- b. Soit on recommence à partir du point 2.





Instructions de répétition

Remarque : les points 2 et 3 peuvent être inversés, en fonction du type de boucle considéré, comme on va le voir plus loin.

Si l'initialisation manque (point 1) alors une erreur d'exécution se produira. Pensez à toujours initialiser une variable.

Il est absolument nécessaire qu'à chaque itération, la valeur de la variable de boucle soit modifiée. En fonction du type de boucle, cette modification est soit intégrée dans l'instruction de boucle, soit à réaliser dans le bloc d'instruction. De plus, la modification réalisée doit impérativement provoquer, au bout d'un nombre fini de répétitions, un changement de la valeur de la condition. Si ces deux contraintes ne sont pas respectées, alors la condition reste toujours la même, et on ne sortira jamais de la boucle.

Boucle infinie : boucle dont la condition ne change jamais, ce qui provoque un nombre infini de répétitions. Il s'agit généralement d'une erreur d'exécution.

Autrement dit : la raison pour laquelle la boucle s'arrête, c'est que la variable de boucle a été modifiée de façon à ce que la condition change de valeur. On emploie le mot itérer pour indiquer qu'on exécute une fois la boucle :

Itération : réalisation d'un cycle complet de boucle. Ceci inclut forcément l'exécution du bloc d'itération de la boucle, mais aussi éventuellement le test de condition, et aussi la modification, en fonction du type de boucle considéré.

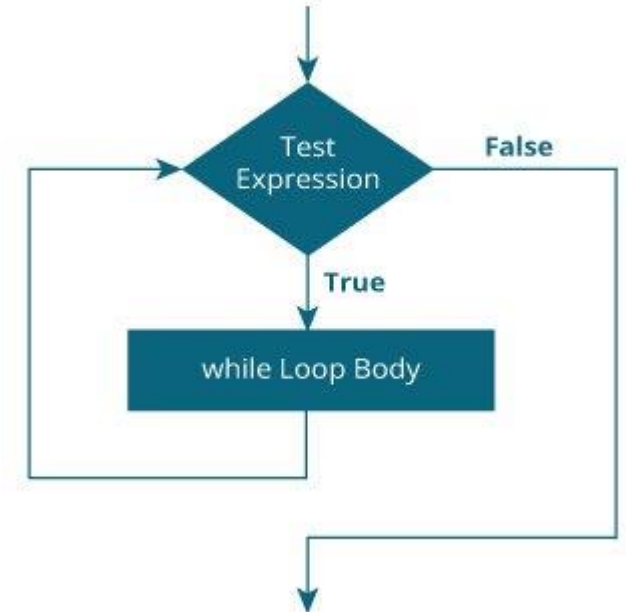


1. Boucle tant que

- La boucle tant que permet de répéter l'exécution d'un bloc d'instructions tant qu'une condition est vraie. Elle repose sur l'instruction `while`.
- La condition est exprimée sous la forme d'une expression placée entre parenthèses juste après l'instruction `while`. La condition est elle-même suivie du bloc d'instructions à répéter.

```
while(condition)
{
    instruction 1;
    ...
    instruction n;
}
```

- Notez l'absence de point-virgule (i.e. `;`) après la condition.





1. Boucle tant que

- Un while fonctionne de la façon suivante. Tout d'abord, la condition est évaluée. Si elle est vraie (i.e. valeur non-nulle), alors le bloc d'instructions est exécuté. La variable de boucle étant supposée être modifiée dans ce bloc, la valeur de la condition est susceptible d'avoir changé. Donc, après avoir exécuté le bloc, on re-teste la condition. Si elle est toujours vraie, on exécute le bloc une autre fois. On répète ce traitement jusqu'à ce que la condition soit fausse, et on continue alors l'exécution du reste du programme.

Remarque : si la condition est fausse dès le début, le bloc n'est même pas exécuté une seule fois.
exemple : considérons le programme suivant :

```
int x;  
...  
x = 0; //initialisation  
while(x<10){  
    printf("x=%d\n",x);  
    x = x + 1; // modification  
}
```

x=0	x=5
x=1	x=6
x=2	x=7
x=3	x=8
x=4	x=9

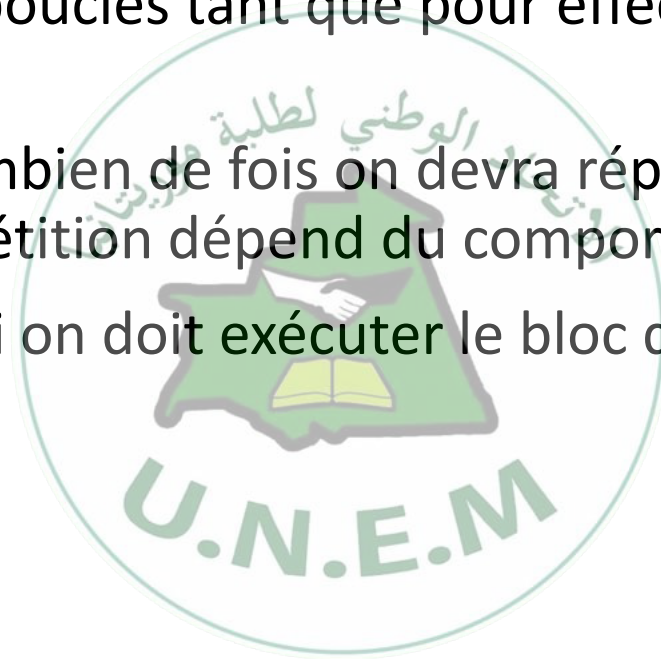
- Si on remplaçait l'initialisation par x=5, on n'aurait que la colonne de droite. Si on la remplaçait par x=20, alors rien ne serait affiché du tout.



1. Boucle tant que

Par convention, on utilise les boucles tant que pour effectuer des répétitions dans le contexte suivant :

- On ne sait pas à l'avance combien de fois on devra répéter le bloc (par exemple parce que le nombre de répétition dépend du comportement de l'utilisateur).
- Et on ne sait pas à l'avance si on doit exécuter le bloc qu'une seule fois.





2. Boucle répéter

- La boucle répéter permet d'exécuter un bloc d'instructions d'abord, de façon inconditionnelle, puis de répéter son traitement tant qu'une condition est vraie. Une boucle de ce type est introduite par l'instruction `do`, suivie du bloc d'instructions. La condition est exprimée sous la forme d'une expression entre parenthèses, placée après un `while`, comme pour la boucle `tant que`. La différence ici, est que le `while` est placé après le bloc, et qu'elle est suivie d'un point-virgule (i.e. `;`). N'oubliez surtout pas ce point-virgule.

```
do {  
  instruction 1;  
  ...  
  instruction n;  
} while(condition);
```

- Un `do...while` se comporte comme un `while`, à l'exception du fait que le bloc sera toujours exécuté au moins une fois, même si la condition est fausse.



2. Boucle répéter

- Son principe de fonctionnement est le suivant. Le bloc d'instructions est exécuté. Puis, la condition est testée. Si elle est fausse, on sort de la boucle et on continue l'exécution normale du programme. Si elle est vraie, on exécute de nouveau le bloc, puis on re-teste la condition. Comme pour le while, il est donc nécessaire d'initialiser manuellement la variable de boucle avant le do. De plus, cette variable doit apparaître dans la condition, et doit être modifiée dans le bloc, sinon la boucle sera infinie.

- exemple : on adapte le programme précédent :

```
int x;  
...  
x = 0; // initialisation  
do {  
    printf("x=%d\n",x);  
    x = x + 1; // modification  
} while(x<10);
```

x=0	x=5
x=1	x=6
x=2	x=7
x=3	x=8
x=4	x=9

- Comme avec le while, si on remplaçait l'initialisation par x=5, on n'aurait que la colonne de droite. Par contre, si on la remplaçait par x=20, alors on aurait l'affichage suivant (alors que while n'affichait rien du tout) :

x=20



2. Boucle répéter

Comme pour le `while`, la boucle `do...while` s'utilise par convention dans une situation bien spécifique :

- On ne sait pas à l'avance combien de fois on devra répéter le bloc.
- On sait à l'avance qu'on doit exécuter au moins une fois le bloc. C'est donc le deuxième point qui est différent, par rapport à la situation décrite pour le `while`.



3. Boucle pour

La boucle pour permet de répéter l'exécution d'un bloc en automatisant les phases d'initialisation et de modification de la variable de boucle. En effet, le mot-clé for, qui précède le bloc, est suivi de trois expressions séparées par des points-virgules :

- La première permet d'effectuer l'initialisation : elle est exécutée une seule fois, au début de la boucle ;
- La deuxième est la condition : elle est évaluée au début de chaque itération ;
- La troisième est la modification : elle est exécutée à la fin de chaque itération.

```
for(intialisation; condition; modification) {  
    instruction 1; ... instruction n;  
}
```

- À noter qu'il est possible d'initialiser/modifier plusieurs variables de boucle en même temps en utilisant des virgules dans les expressions :

```
for(x=1,y=10; x=y; x++,y--)
```



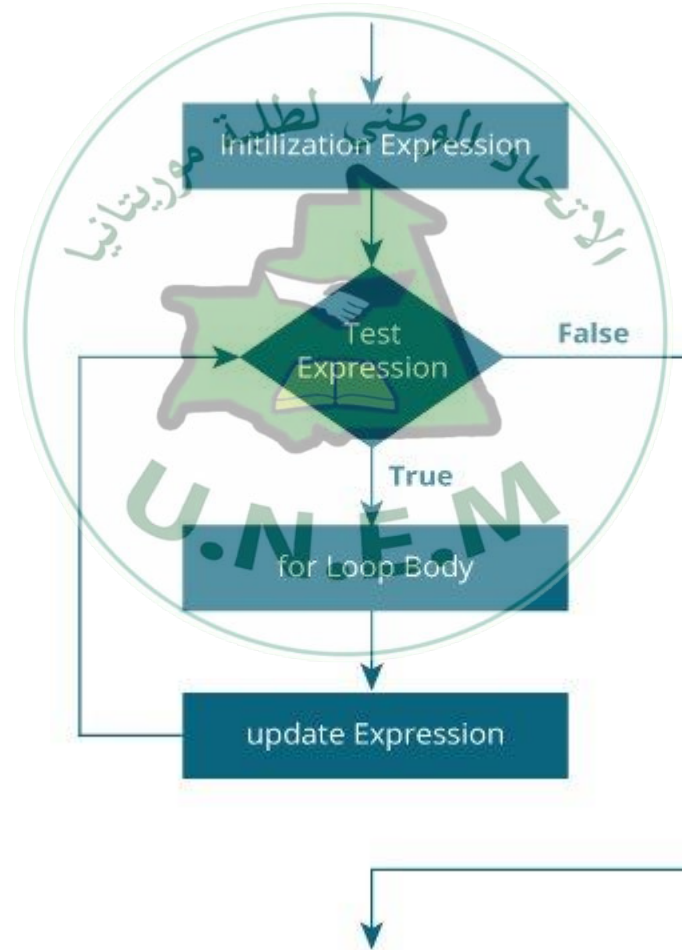
3. Boucle pour

Le principe de fonctionnement du for est le suivant. Tout d'abord, à la différence de `while` et `do...while`, l'initialisation est intégrée à l'instruction. Mais comme pour les autres boucles, cette opération est réalisée une seule fois, au tout début du traitement de la boucle. De plus, il reste nécessaire de déclarer la variable avant la boucle.

Après l'initialisation, la condition est évaluée. Si elle est vraie, alors le bloc d'instructions est exécuté. À la différence de `while` et `do...while`, il ne faut surtout pas modifier la variable de boucle dans ce bloc. En effet, pour le for la modification est intégrée à l'instruction. Cette modification est effectuée automatiquement juste après que le bloc a été exécuté.



3. Boucle pour





3. Boucle pour

Après exécution du bloc et application de la modification, la condition est évaluée. Si elle est toujours vraie, on recommence le même traitement (bloc puis modification puis condition). Sinon, on sort de la boucle.

exemple : programme équivalent au précédent basé sur `while` et `do...while` :

```
int x;  
...  
for(x=0;x<10;x++) {  
    printf("x=%d\n",x);  
}
```

L'affichage sera le même que précédemment. Si on initialise `x` à 5 dans la boucle, l'affichage sera là-aussi le même. Si on initialise `x` à 10, alors la condition sera fausse lors du premier test, et le bloc ne sera pas exécuté (comme pour le `while`, et à la différence du `do...while`).

Nous utiliserons les boucles pour uniquement quand on saura à l'avance combien d'itérations devront être effectuées. Il est interdit d'utiliser un `for` quand le nombre de répétitions ne peut pas être prévu.



4. Break et Continue

```
while (testExpression) {  
    // codes  
    if (testExpression) {  
        continue;  
    }  
    // codes  
}
```

```
do {  
    // codes  
    if (testExpression) {  
        continue;  
    }  
    // codes  
} while (testExpression);
```

```
while (testExpression) {  
    // codes  
    if (condition to break) {  
        break;  
    }  
    // codes  
}
```

```
do {  
    // codes  
    if (condition to break) {  
        break;  
    }  
    // codes  
} while (testExpression);
```

```
for (init; testExpression; update) {  
    // codes  
    if (testExpression) {  
        continue;  
    }  
    // codes  
}
```

```
for (init; testExpression; update) {  
    // codes  
    if (condition to break) {  
        break;  
    }  
    // codes  
}
```